Agentic Lab

Architecture: roles, logic, conventions

Framework reference, as of the merge of PR #13

October 2026

Abstract

Agentic Lab is a framework in which a small team of humans and AI agents run a scientific lab together. Members talk to their *manager* agents in Slack; managers delegate work to *worker* agents on GPU machines; a *secretary* keeps the lab running. Every agent runs Claude (through the Claude Agent SDK) on a human member's Claude seat, inside Claude Code's built-in sandbox, as its own Unix user. Anything an agent can't do inside its sandbox is asked of a human in a Slack DM. The lab is described by one file, `lab.yaml`, kept in the lab's private configuration repository; only what is merged to that repository's `main` is ever deployed. This document describes the roles, the mechanisms that bind them together, and the conventions the framework relies on.

Contents

1 Principles **3**

2 Roles **3**
- 2.1 Humans . . . 3
- 2.2 Agents . . . 3
- 2.3 Services that are not agents . . . 4
- 2.4 Hosts . . . 4

3 The big picture **4**

4 Identity, users and tokens **5**

5 The sandbox and permissions **6**
- 5.1 Inside the sandbox: no questions . . . 6
- 5.2 Outside the sandbox: a human decides . . . 6

6 Work: managers and workers **7**
- 6.1 Workers . . . 7
- 6.2 Managers . . . 8
- 6.3 Who may use what . . . 8

7 The secretary **9**
- 7.1 State and the web UI . . . 9
- 7.2 Claude usage . . . 9
- 7.3 Machines . . . 9

7.4 Notices 10

8 Configuration: lab.yaml and the config repository **11**
8.1 lab.yaml 11
8.2 Only reviewed configuration is deployed 11
8.3 What lab apply does 11

9 Administering the lab from Slack **12**

10 Secrets **13**

11 GitHub for agents **13**

12 Observability **14**

13 Security summary **14**

14 Conventions **14**

15 Command reference **15**

16 Not in this version **15**

1 Principles

1. **Humans decide, agents do.** Agents never approve anything: not each other's actions, not their own, not pull requests. Approvals come from the human the action is for, in a Slack DM, from a dedicated app.
2. **Sandbox the environment, not the commands.** There is no command parsing or classifier. Inside Claude Code's sandbox an agent may do anything; leaving it (the internet, writes outside its folders) needs a human.
3. **Secrets belong to services, not to agents.** Each agent's service holds its tokens in a folder the agent's sandbox can't read. Secrets never pass through a chat: they arrive through one-time links and hidden prompts.
4. **One file describes the lab, and git is its history.** `lab.yaml` declares humans, agents, hosts and grants. Every change is a reviewed pull request in the lab's private configuration repository; deployment takes only `main`.
5. **Every agent is attached to a human, in terms of tokens.** An agent runs on one member's Claude seat. Who may *use* an agent is a separate setting.
6. **Transparency over limits.** No spend limits; instead the web UI and the secretary show usage per Claude account, with when and from where each number was read.
7. **Simple, generic, portable.** No hard-coded hosts, organisations or project tools. Experiment trackers and similar tools are extensions a lab adds itself.

2 Roles

2.1 Humans

Member A person in `lab.yaml` (`humans:`), with a Slack member id. Talks to managers and the secretary in Slack, receives the approvals for work done for them, and lends their Claude seat to the agents attached to them.

Admin A member with `admin: true` and a `tailscale_login`. Only admins can read agents' logs in the web UI, change the lab (through the secretary or the console), deploy, power a machine that isn't dedicated to them on or off, or ask for someone else's token.

2.2 Agents

Secretary Exactly one per lab, on an always-on host. Keeps the lab running: collects its state, serves the web UI, powers cloud machines on and off (and stops idle ones), warns about expiring tokens, answers questions about the lab in Slack, and is how admins administer the lab after the install (§9). It refuses research work and points to the member's manager.

Manager A member's team-mate in Slack: it understands requests, delegates work to workers, follows the tasks and reports back with summaries, plots and files. It never approves; it relays. A manager is *dedicated* (serves one member) or *shared* (serves every member, with a separate memory for each).

Worker An agent on a GPU machine that takes tasks over MCP, one at a time, first come first served, and reports progress and a result. Also *dedicated* or *shared*.

2.3 Services that are not agents

Broker On the secretary host, as `lab-broker`. Receives every permission request from every agent, works out which human decides, and asks them through the *Lab Approvals* Slack app. Holds active grants.

Intake On the secretary host. Receives secrets through one-time codes: web forms (`/s/<code>`) and a small client script for Claude tokens.

2.4 Hosts

- **Secretary host**: small, always on (on-prem or a small VM). Runs the secretary, broker, intake and web UI, and the admin's Claude Code session for the install.
- **Managers** run on any host: the secretary host, a CPU VM of their own, or a GPU VM next to the workers they use. Each manager's `host:` in `lab.yaml` says where; several agents on one host still each have their own Unix user.
- **GPU hosts**: cloud VMs (Nebius in this version), stopped once idle for `idle_stop_min` minutes (required on cloud hosts; 0: never).
- **AWS EC2** for ordinary hosts (the secretary, managers): `provider: aws`, powered like Nebius VMs (below); a public IPv4 with a security group allowing no inbound (a VM without one needs a costly NAT gateway), and an EBS root volume (instance store is wiped on every stop).
- All hosts are on one **Tailscale tailnet**; services talk to each other only over it.

3 The big picture

A typical piece of work:

1. A member DMs their manager: *"train the baseline on worker-a and show me the loss curve"*.
2. The manager submits a task to the worker over MCP, naming the member as the requester, and registers it with the broker.
3. The worker runs the task in its sandbox, reporting progress. If it needs something outside the sandbox (say, a dataset from the internet) it asks; the broker DMs *the requester* with buttons via the *Lab Approvals* Slack app; the answer comes back to the worker.
4. The manager follows the task and posts progress, the plot and a summary in the member's thread. The secretary's web UI shows it all as it happens.

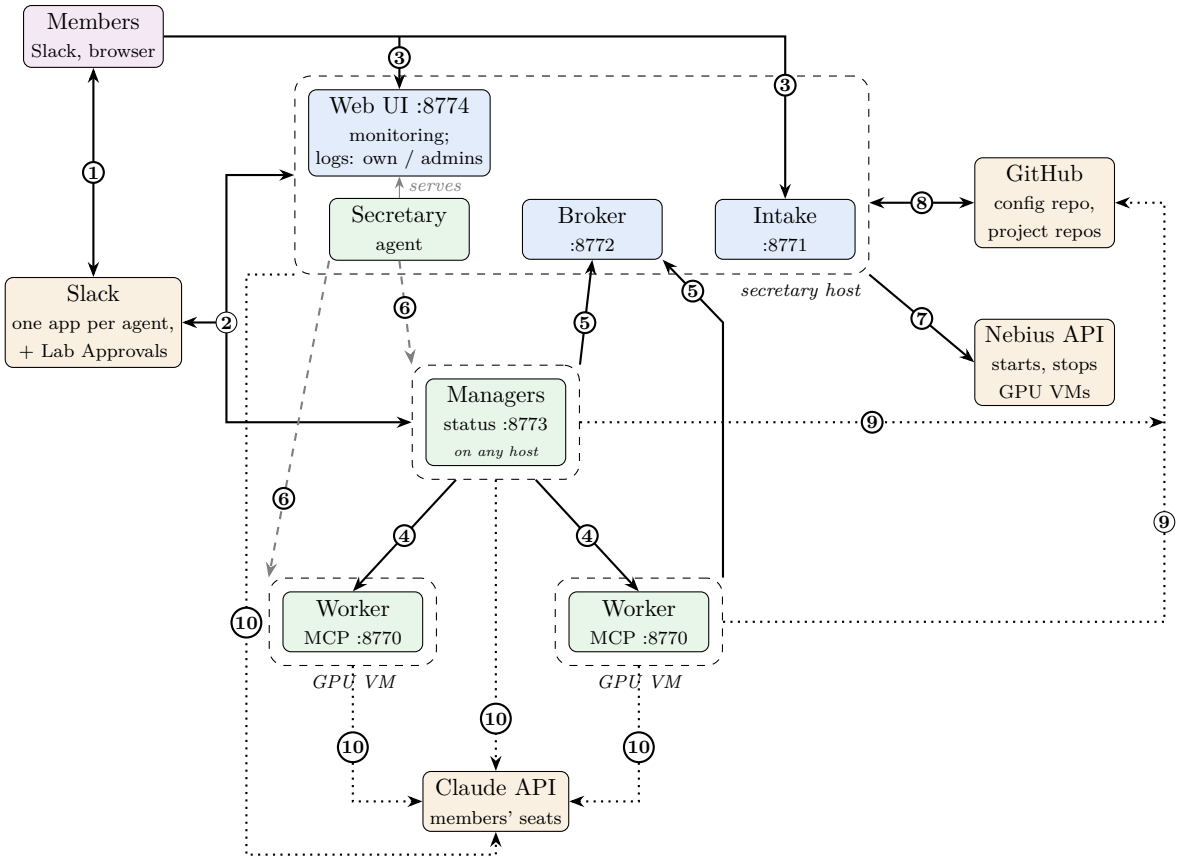


Figure 1: Components and connections. Dashed boxes are hosts; all traffic between lab hosts runs over the tailnet. Slack, GitHub, Nebius and Claude are reached over the internet by the services, never by sandboxed agent commands, except GitHub for an agent with a bot (9).

The numbered connections in Figure 1:

1 Members talk to agents and answer approvals in Slack (DMs, threads, buttons).
2 Each Slack-facing agent has its own Slack app (the secretary, every manager), the broker has *Lab Approvals*; all connect *out* to Slack (Socket Mode, no public URL).
3 Over the tailnet: members' browsers open the web UI (served by the secretary; monitoring only; agents' logs for their members, all of them for admins); their laptops hand secrets to the intake (one-time links and codes).
4 Managers give workers tasks and exchange files with them (MCP, a token per pair).
5 Every worker and every manager sends its permission requests to the broker, which asks the human the action is for (2, 1).
6 The secretary monitors workers (MCP, read-only token), managers (status endpoint) and the broker; `lab apply/apply-main` deploy from this host to every host over SSH.
7 The secretary starts and stops the GPU VMs through the Nebius API.
8 Deploys fetch the config repo's `main`; the secretary pushes branches for review.
9 Managers and workers with a GitHub bot reach github.com from their sandboxes.
10 Every agent (the secretary too) runs Claude on its member's seat.

4 Identity, users and tokens

- **One Unix user per agent**: `lab-<agent id>` (a system user, no login shell, no SSH keys, home folder mode 700). Several agents on one host never share a user, so they can't read

each other's files. Humans log in as themselves.

- Each agent runs as a **systemd service** under its user: `lab-worker@<id>`, `lab-manager@<id>`, `lab-secretary`; plus `lab-broker` and `lab-intake`.
- **Folders per agent user**:
  - `~/work`: its working folder (writable in its sandbox);
  - `~/.config/agentic-lab`: its service's secrets (Claude token, Slack tokens, lab tokens, GitHub token), *not readable* by its sandbox;
  - `~/.local/state/agentic-lab`: the service's state (task queue, memory, usage, event log), *not readable* by its sandbox.
- **Claude**: `human:` on an agent says whose Claude seat it runs on (a `claude setup-token` token, valid a year). The token reaches Claude Code as an environment variable that is stripped from sandboxed commands.
- **Lab tokens**: generated by `lab apply` and never shown; each pair of services that talk has its own (manager→worker, agent→broker, the secretary's *read-only* monitoring tokens). They live in `/etc/agentic-lab/private/tokens.yaml` (root) and in the services' secret folders.
- **Display names**: every agent has an id (`mgr-flash`) and a `display_name` (*Flash*), the name humans see. Names are the admin's choice; the framework and its guide never invent them.

5 The sandbox and permissions

5.1 Inside the sandbox: no questions

Every agent's Claude Code runs with the built-in sandbox (bubblewrap + seccomp), rendered per agent from `lab.yaml` (`lab settings <agent>` prints it):

- **Network**: only the lab's tailnet names (`*.<tailnet>`), plus hosts granted to it; strict allowlist (unknown hosts are refused, not prompted). Raw IPs are refused: lab resources are reached by their tailnet names.
- **Files**: writes only in the working folder, `/tmp` and granted paths; reads anywhere except `/run`, `/var/run` and its service's secret and state folders.
- **GPU**: workers see the GPU through a wrapper around bubblewrap (`bwrap-gpu`, set as `sandbox.bwrapPath` in Claude Code's managed settings) that binds the NVIDIA devices; Unix sockets are allowed for CUDA, and `/run` is hidden instead (closing a `systemd-run` escape).
- **Host setup** (`setup-sandbox.sh`): an AppArmor profile that lets bubblewrap create its namespaces on Ubuntu, the GPU wrapper, the managed settings.

Any Bash command that stays in the sandbox is allowed without asking. Caches (`XDG_CACHE_HOME`) point into the working folder. Managers auto-compact their context at 17% of the window (`compact_percent`); others at Claude's default.

5.2 Outside the sandbox: a human decides

When the sandbox refuses something, the agent sees why and may retry outside it. That retry is a permission request: the agent's service sends it to the broker, with the sandbox violations that

explain it.

Who decides The human *the action is for*: a worker task's requester (as registered by the manager that submitted it, not as claimed by the worker); for a manager, the member whose message started the work. Never an agent.

How A DM from the *Lab Approvals* app with buttons: **Run once**, **Allow *X* for 1 hour**, **Allow *X* for this task**, **Deny**. A reply in the DM's thread denies, with the reply as the reason.

Timeout Off by default: a request waits until the human answers. A member may set one (`timeout_s` in `lab.yaml`); then a request nobody answers in time is closed as *denied*: silence counts as "no", never as "yes" (`on_timeout` can only be `deny`), and the agent is told it was a timeout, not a refusal. A member changes their own timeout from Slack, through the secretary (an admin anyone's; it applies to new requests). If the agent stops waiting (its task is cancelled, or a manager's turn is cut off), the request is *withdrawn*, so nobody is asked about it any more.

Grants "For a while" answers become *grants* that widen the sandbox (a domain, a writable path, or a tool) with an expiry; the agent's session restarts with the wider sandbox. *Standing* grants are written in `lab.yaml`. "Trust everything unsandboxed for an hour" does not exist.

Managers relay: they can show members what their tasks are waiting for and add context to a pending request; they cannot decide it.

6 Work: managers and workers

6.1 Workers

- An MCP server on the tailnet (`http://<host>:8770/mcp`), authenticated by bearer tokens: one per manager allowed to use it, plus the secretary's read-only one.
- Its tools:
  `submit_task`, `task_status`, `task_result`, `cancel_task`, `list_tasks`, `get_guardrails`, `read_file` and `write_file` (its working folder, up to 20 MB), `health_check` (queue, long runs, GPU and its processes, disk, usage), `check_usage`, and `recent_events` (its log: *the secretary's token only*).
- Tasks run one at a time, first come first served; a restart requeues a running task and resumes its Claude session. No fixed results contract: a result is a summary plus whatever artifacts fit (files, commits, links).
- **Long runs** (training for hours): nothing the agent starts in its session outlives it (a sandboxed command's processes die when it returns, the rest with the session or a service restart). So the agent calls `start_run`; a human approves *each* run, since it outlives the session with its network not limited (its file rules are the agent's own), as the agent's user, as its own unit `lab-run-<agent>@<id>`, with its output in a log in the working folder. With `wait_for_runs` the task becomes `waiting`: still in progress, parked, and the worker takes other tasks meanwhile. It resumes when a run ends or at a check-in time, with the runs' state and log tails; cancelling it stops its runs. Runs survive deploys and worker restarts.
- **Warm environments**: tools' caches (uv, pip, torch extensions, Triton, Hugging Face, CUDA) go to `.cache/` in the working folder and stay between tasks; checkouts in `repos/`,

environments in `envs/`, listed in `ENVIRONMENTS.md`, reused before rebuilding. Below `keep_free_gb` of free disk the worker prunes them and says what it removed.

6.2 Managers

- **Slack**: Socket Mode (no public URL). A DM is one conversation; in channels a manager joins only threads where it's mentioned or already involved, reading the thread so far on first mention: its first 5 messages (they often set the topic) and the last 50 before the mention. In such a thread it sees every message and may stay silent. Attachments (up to 50 MB) are downloaded into its working folder.
- Every message is checked against `lab.yaml`: only lab members are answered. On a dedicated manager, other members are *guests* (§6.3).
- **Tools**: list workers, submit and follow tasks, pending approvals and notes on them, post files, fetch a worker's file into the conversation, send a file to a worker, `recall` and `remember`.
- **Memory**: durable notes about members and their work. A dedicated manager has one memory; a shared manager keeps one per member, separated by code (private conversations stay private).
- **Follow-up**: a background loop posts each task's progress and result in the conversation that started it, mentioning the requester.
- **Staying reachable**: a manager runs one turn at a time *per conversation* (other threads and members are never blocked). It does work itself only when it takes a few minutes (an issue, a comment, a small fix); longer work and anything needing a GPU goes to a worker. A turn longer than `turn_budget_min` (default 5) is interrupted, and the manager gets one short turn (90 s) to say where it got to or hand the rest to a worker. A message arriving while it's busy in that conversation gets an immediate "still working on the previous message" reply.

6.3 Who may use what

Every manager and worker has an `access` setting in `lab.yaml`: `dedicated` (the default) or `shared`. "Using" means something different for each role:

	`access: dedicated` (default)	`access: shared`
Manager (members talk to it)	Its own member (the one in `human:`) gets everything. Every other member can talk to it as a *guest*: questions, explanations, scientific discussion, but no work (see below).	Every member of the lab gets everything. It keeps a separate memory for each member.
Worker (managers give it tasks)	Only the managers of its own member can give it tasks.	Every manager in the lab can give it tasks; they queue first come, first served.

Guests of a dedicated manager only talk with it. In a guest's conversation, code (not the prompt) takes away every tool: no commands or files, no worker tasks, no GitHub (its bot's token

is left out), nothing outside the sandbox, and no memory (the member's private notes stay private). A guest's turns are cut off after 2 minutes. Asked for real work, the manager says whose manager it is and points to the guest's own manager or a shared one. In the owner's thread, a guest's turn never resumes the owner's session: it starts afresh from the thread as everyone sees it, and the owner's next turn gets a short "meanwhile in this thread" note.

`access` only says who may use an agent. It doesn't change whose Claude seat the agent runs on: that's always its `human:`, whoever asked. A shared worker owned by Alice runs Bob's task on Alice's seat. Approvals still go to the member the work is for (Bob).

7 The secretary

7.1 State and the web UI

Every 15 s the secretary collects the lab's state: workers (over MCP, with its read-only token), managers (their status endpoint), the broker (pending approvals, grants), local services (systemd) and machines (Nebius, every few rounds). It serves:

- **The web UI** (`http://<secretary host>:8774/`, tailnet only, refreshes every 20 s): Claude usage per account, workers and their tasks, approvals and grants, members' approval timeouts, machines, agents, and which configuration and framework commits are deployed.
- **Agents' logs** (`/logs/<agent>`): admins see all; a member sees their dedicated agents' whole logs and their own entries of shared agents and the secretary. The request's tailnet address must not be a lab host; `tailscale whois` must name a person (not a tagged device) whose login is a member's in `lab.yaml`. Tailscale authenticates every packet with the device's key, so the address can't be forged.
- **A Slack Home tab**: a personal digest (your usage, your tasks, approvals waiting for you, machines) with a link to the web UI.

7.2 Claude usage

- Each agent records its tokens per run (new vs. cached) and its seat's limit readings. Readings are per account (a seat is shared by all its agents); the freshest wins, and every number shows *as of when* and *from which agent*.
- Percentages come from a **probe**: every API reply carries the seat's 5-hour and weekly utilization in headers, which Claude Code doesn't pass on. Each agent's service sends one tiny request (Haiku, 1 output token) every 10 minutes, only while its seat's 5-hour window is known to be open, so it never opens one.
- **Check now** on each account in the web UI asks one agent on that seat to probe at once (once a minute at most).

7.3 Machines

- **Power**: the secretary starts and stops the cloud VMs listed in `lab.yaml` (`kind: cloud`, `provider`, `instance`): on Nebius with a service account that may only view, start and stop VMs; on AWS with an access key limited to the lab's tagged instances (never an instance role). Never its own host.

- **Busy** means: a worker there has a task running, queued, awaiting approval or waiting on its long runs, or a long run going (even CPU-only); or any process on a GPU (anyone's, not only the lab's); or a GPU above 0% in any of its last 5 readings (about 75 s: a GPU counts as idle only after 0% in several readings in a row); or someone keeps it awake with a `~/.lab-keep-awake` file (below); or a manager runs there.
- **Working on a machine directly**: the lab recommends talking to agents, but anyone may still log in and work on a lab machine themselves, e.g. a CPU-only job the GPU checks can't see. To make sure the machine isn't stopped under them, they create `~/.lab-keep-awake` in their home (its first line, optional, says why). While any such file exists the machine counts as busy; the web UI shows who keeps it awake. A root timer on every lab host lists these files each minute (home folders are private, so the lab's services can't look themselves).
- **Idle stop**: a host with workers is stopped after its `idle_stop_min` minutes (if above 0) of not being busy. It's judged by its workers alone: a manager there goes offline in Slack until the host is powered on. A manager's own work isn't measured, so `idle_stop_with_managers: false` makes managers keep the host busy A host with managers and no worker counts as busy (it's never idle-stopped; powering it off takes `force_off`).
- **Who may power**, on and off alike: a machine dedicated to a member (all its agents theirs and dedicated), that member or an admin, busy or not; any other machine (a shared agent, agents of more than one member, the secretary, or no agents), an admin only. A busy machine goes off only after confirming (`force_off`).

7.4 Notices

Daily, the secretary warns about credentials and GitHub bot tokens expiring within 14 days (from `lab.yaml`) and can post lab-wide notices to an `announcements` channel.

8 Configuration: lab.yaml and the config repository

8.1 lab.yaml

Section	Contents
`lab`, `tailnet`	The lab's name; the tailnet's DNS suffix (`trust_tailnet`: whether sandboxed commands may reach it; default yes).
`humans`	Members: `name`, `slack_user`, `channels` (permissions: Slack DM), `admin`, `tailscale_login`.
`agents`	Each agent: `role`, `human` (whose seat), `host`, `access`, `display_name`, `grants` (standing), `github` (its identity, or several by repository), `model`, `workdir`, `gpu` (default: workers yes), `port` (workers, managers), `compact_percent`, `slack_app`, `turn_budget_min` (managers), `keep_free_gb` (workers: free disk below which it prunes its caches and environments; default 30).
`hosts`	`kind` (onprem/cloud), `provider` (nebius, aws), `instance`, `idle_stop_min` (required on cloud hosts; 0: never), `readers` (Linux users who may read its agents' working folders), `tailnet_name`, `ssh_user`, `idle_stop_with_managers`.
`github`	`accounts`: the agents' identities: bot accounts (commit `email`, token `expires`) and GitHub Apps (`app_id`, `email`, and `installation_id` when the app is installed in several places).
`credentials`	Other expiring credentials to be reminded of.
`announcements`	A Slack channel for lab-wide notices.

`lab check` validates everything at once (unknown keys are errors) and warns about what is merely odd. Agent ids are lowercase (`[a-z][a-z0-9-]*`), since they become Unix user names.

8.2 Only reviewed configuration is deployed

- The lab's configuration lives in its own **private repository**: `lab.yaml` and `framework.lock`, which pins the framework to one commit. The framework repository is only read.
- `lab apply` fetches the config repository and deploys `lab.yaml` and `framework.lock` *as they are on* `origin/main`, never the working copy. It ships the framework only from a clean checkout at the pinned commit.
- Every change: a branch → `lab plan -working-tree` (preview) → a pull request (`lab pr-link`) → a human reviews and merges → `lab apply`. Upgrading the framework is the same: `-update-framework` only rewrites `framework.lock`; the new pin goes through a pull request.
- Every host records what it runs in `/etc/agentic-lab/applied.yaml` (config and framework commits, when, by whom), shown by `lab doctor` and the web UI.
- **Break glass**: `lab apply -unreviewed` deploys the working copy; every host flags it as unreviewed until a reviewed deploy replaces it.

8.3 What lab apply does

From the secretary host, as root, for each host (locally, or over SSH as the admin with passwordless sudo): ship the framework to `/opt/agentic-lab` (and `lab` on the PATH), install `lab.yaml`, set

up the sandbox, create the agents' users, install their secrets and generated tokens, install and (re)start their services. Services whose secrets haven't arrived yet are installed but held. Every step is idempotent: re-running is safe. `lab doctor` checks a host against `lab.yaml` and says how to fix each failure.

9 Administering the lab from Slack

After the install, admins work with the secretary in a DM; the console is for the install and emergencies. Admin-only tools, checked in code (the person's Slack identity must be an admin's in `lab.yaml`):

`propose_lab_change` writes the whole new `lab.yaml` on a branch of the secretary's own clone of the config repository, checks it, pushes it, opens the pull request if the secretary has a GitHub identity (else gives the link to open it), and replies with the diff: *"here is the PR, review and merge it"*. It never touches `main`.

`offer_deploy` after the merge, posts a **Deploy** button listing what is merged but not deployed. Only an admin's click deploys; a button left behind by a newer `main` refuses. The result is posted back in the conversation.

`request_slack_app`, `request_github_token` post the steps (manifest, links, clicks) and a one-time link for a new manager's Slack app or a bot's token. With a Slack app comes its icon, drawn by Claude from the agent's name, role and the lab's name (as SVG, converted to PNG; a pattern made from the name if that fails); `redraw_app_icon` draws another.

`request_github_app`, `check_github_app` the steps to create a GitHub App as an agent's identity and a one-time link for its private key; then check that the app works and give its `lab.yaml` lines.

`check_slack_app`, `read_lab_config` check a Slack app's tokens; read the configuration on `main`.

The secretary's one root command is `lab apply-main`, allowed by a sudoers rule with no arguments: it deploys only the config repository's `main`, moving the framework checkout to the pinned commit, as its own systemd unit (so it survives the secretary's own restart). Even a misled secretary can only deploy what a human merged. It reaches the config repository with a deploy key (write: it pushes branches, never merges) or, if it has one (`github:` on the secretary), as its GitHub identity over HTTPS: then `lab apply` makes no deploy key and the pull requests come from the lab's bot. The key or token and its clone live in folders its sandbox can't read, and every commit it makes carries a `Lab-Agent:` trailer.

Still at the console: the install itself; creating the first Slack apps; giving a new GPU VM SSH access; removing an agent's Unix user; `-unreviewed`; uninstalling (`docs/uninstall.md`).

10 Secrets

Secret	How it arrives
A member's Claude token	The secretary DMs them a one-time command. On their own computer it runs `claude setup-token` and sends the token straight to the intake over the tailnet; the intake checks it with Anthropic. Fallback: a hidden prompt.
A Slack app's tokens	A one-time web form (bot token `xoxb-`, app token `xapp-`); `lab slack-verify` checks them with Slack.
A GitHub bot's token	A one-time web form; the intake checks with GitHub that it is that bot's.
A GitHub App's private key	A one-time web form (the app's `.pem`); `check_github_app` / `lab github-app-check` check the app with GitHub.
Nebius service account	`lab nebius connect` (key generated on the host), or a form.
Lab tokens	Generated by `lab apply`; never shown.

One-time codes expire after 24 h and are used once; failed attempts are rate-limited. Rules: no secret in any chat (Slack or Claude); agents can't read their service's secrets; an agent asked to recite a token refuses.

11 GitHub for agents

- A manager or worker may have a GitHub identity (`github: <name>` on the agent), or **several, picked by repository**: for example a lab identity by default and the open-source project's bot for that project, `github: {acme-flash: default, spikyclaudebot: [anatoli-starostin/spiky]}`. git picks the identity by the repository by itself (the token through a credential helper, the commit author through git's conditional configuration); for `gh` on a mapped repository the agent passes that identity's token. Several agents may share an identity. Identities are named in `github.accounts`; two kinds, side by side:
  - **A GitHub App** (`app_id`): appears as `<slug>[bot]`, with no account or email to create, which makes it easy to give each agent one. Its tokens last an hour: the agent's service holds the app's private key (the agent can't read it), mints a token, keeps a fresh one where the agent's git and `gh` read it, and mints the next before it expires.
  - **A full bot account** (a GitHub user with its own email): a fine-grained token in the agent's environment (`GH_TOKEN`), its expiry date tracked.
- Either way git authenticates through a credential helper, commits carry the identity's name and email, `gh` is installed, and `github.com` is reachable from the agent's sandbox.
- The agent *can* read its current token. What keeps this safe is GitHub: private repositories, a fine-grained token limited to the lab's repositories (contents, pull requests, issues), and **branch protection** on `main` (pull requests only, reviews by human code owners, no bypass for bots or apps). Bots push branches and open pull requests, issues and comments; their approval never counts and they can't merge.

12 Observability

- **Event logs**: each agent's service writes its own log (messages, replies, tool calls, approvals, task events), one JSON-lines file per day, kept 14 days, in its state folder. The agent can't read it.
- **Reading logs**: the web UI (admins; members their own agents), `sudo lab logs <agent>` on its host. Workers hand their log only to the secretary's monitoring token; managers' status endpoint likewise.
- **Debugging**: `sudo lab debug <agent>` stops its service and opens its latest session in the classic Claude Code console, as its user with its sandbox; `lab manager ask` and `lab secretary ask` talk to them without Slack.

13 Security summary

Boundary	What enforces it
Agent vs. the internet and the host	Claude Code's sandbox (strict network allowlist, write allowlist, hidden secrets and `/run`); leaving it needs a human.
Agent vs. agent	One Unix user per agent; lab tokens per pair of services; a worker accepts only its allowed managers.
Who approves	The broker asks the human the action is for; timeouts deny; agents have no way to approve.
Who changes the lab	Merged pull requests on the config repository; `lab apply` and `apply-main` deploy only `main`; Slack admin tools check the admin in code.
Who reads logs	Tailscale identity matched to `admin: true`; lab hosts and tagged devices refused.
Secrets	Services' folders unreadable by sandboxes; one-time intake; no chat.
GitHub	Private repos, fine-grained tokens, branch protection.

Residual risks, by design: anyone with `sudo` on a lab host is effectively an admin there; an admin's device and Tailscale login are the keys to the logs; the web UI's main page is visible to the whole tailnet (restrict it with Tailscale ACLs); a GitHub bot's token is readable by its agents.

14 Conventions

- **Names**: agents' display names are chosen by the admin, never invented; ids derive from them (`Flash` → `mgr-flash`); Unix users are `lab-<id>`; services `lab-<role>@<id>`.
- **Ports** (tailnet only): worker MCP 8770, intake 8771, broker 8772, manager status 8773, secretary web UI 8774.
- **Paths**:
  - framework: `/opt/agentic-lab`; deploy logs: `/var/log/agentic-lab`;
  - configuration: `/etc/agentic-lab` (`lab.yaml`, `applied.yaml`, `private/`);

  - per agent: `~/work`, `~/.config/agentic-lab` (secrets), `~/.local/state/agentic-lab` (state).

- **Changes**: branch → pull request → human merge → deploy, for the lab's configuration and the framework alike.
- **Slack apps**: one per Slack-facing agent (each manager, the secretary) and one *Lab Approvals* app for the broker; manifests generated by `lab slack-manifest`; created one at a time, verified before the next.
- **Guidance**: the admin's own Claude Code session follows the repository's `lab-admin` skill and `docs/install.md`: explain before acting, ask rather than assume (names, ids), derive rather than ask (App IDs, tailnet names), never take a secret in the chat.

15 Command reference

Command	Purpose
`lab init`	Write the first `lab.yaml` (and `framework.lock`).
`lab check`	Validate `lab.yaml`.
`lab plan [-working-tree]`	What a deploy would do; missing secrets.
`lab apply [-update-framework] [-unreviewed]`	Deploy `main` (sudo).
`lab apply-main`	The secretary's deploy of `main` (sudoers).
`lab pr-link`	The link to open a pull request for the config branch.
`lab doctor`	Check this host; every failure says how to fix it.
`lab secret request`	A one-time code or link for a secret.
`lab slack-manifest`, `lab slack-verify`	A Slack app's manifest; check its tokens.
`lab nebius connect`	Set up VM power with one admin login.
`lab settings <agent>`	The agent's rendered sandbox settings.
`lab logs`, `lab debug`	An agent's log; its session in the console.
`lab manager ask`, `lab secretary ask`	Talk to them without Slack.

16 Not in this version

- Per-token (API) billing: agents run on members' Claude seats only.
- Permission channels other than Slack DM; approvals in the web UI (it monitors only).
- Picking a GitHub identity per task or per member (identities are picked by repository).
- VM power for cloud providers other than Nebius (AWS hosts work, unpowered).
- An uninstall command (`docs/uninstall.md` describes the steps).
- Project tools such as experiment trackers: extensions a lab builds itself.